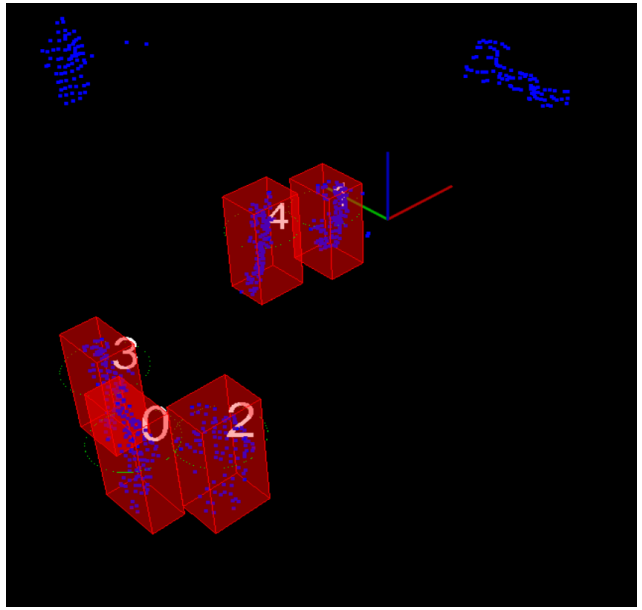
Kalman Filter Assignment

Nacho Sañudo
HIPERT - University of Modena and Reggio Emilia
isanudo@unimore.it

Academic year: 2026/2027

Objective

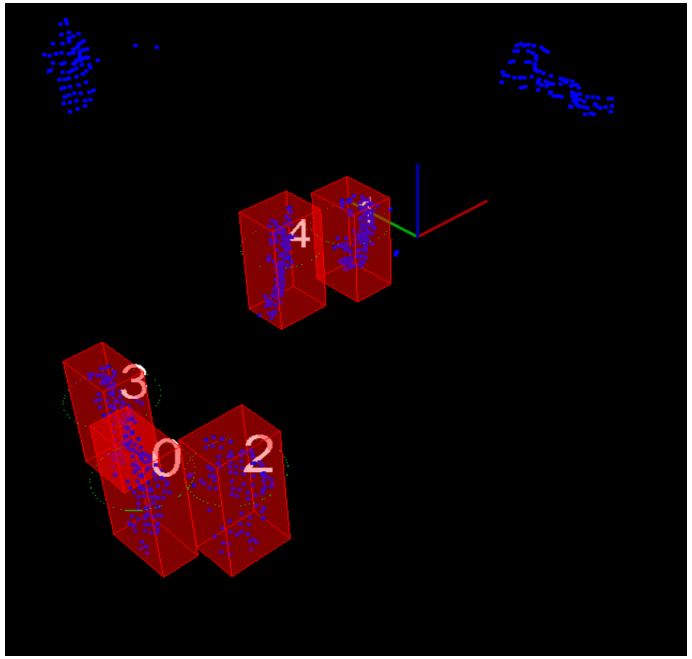
- Build a multi-object tracker
- Input: point cloud in a dataset with multiple clouds
- Output: Tracked objects
  - Each object is unique
  - To detect the position of the objects we will use the clustering technique
    - The output of the clustering is the position x,y
- Objective: Apply a Kalman filter to estimate the position [x,y]
    - we can estimate the velocity and even acceleration of an object with the measurement of its locations
    - We can track the obejcts even if we miss measurements



sudo apt install libeigen3-dev

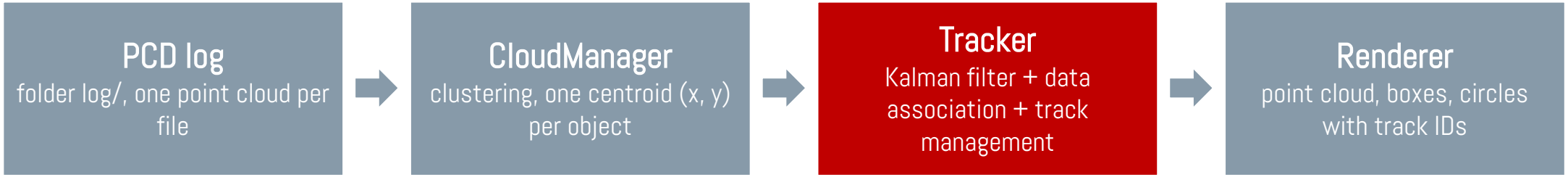
Multi object tracker

- Tracking multiple objects poses different challenges:
  - Multiple detections must be associated with the correct tracks
  - You must handle new objects appearing in a scene
    - Old objects too!
  - Object identity must be maintained when multiple objects merge into a single detection
  - Object occlusion…



Objective

Build a **multi-object tracker**: every pedestrian in the LiDAR log must keep a **unique ID** over time, also when its detection is missed for a few frames



Challenges

› associate each detection with the right track

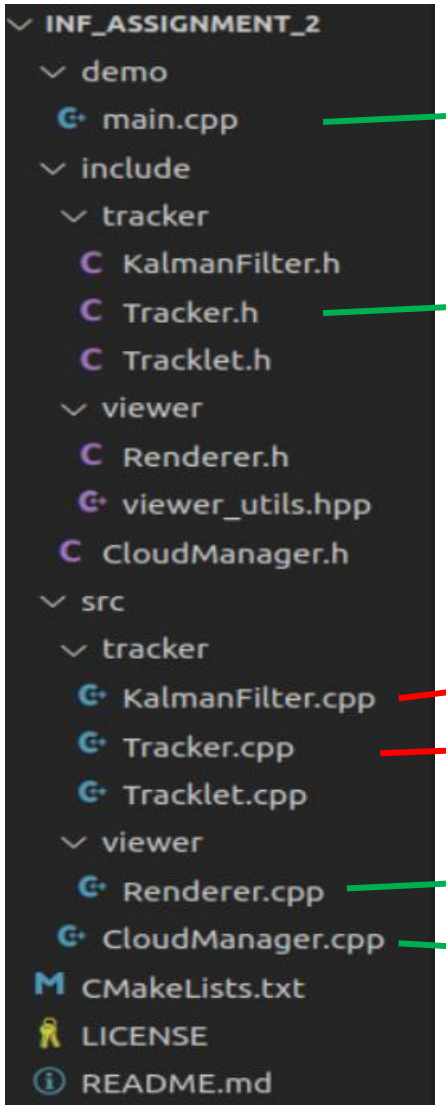
› new objects entering, old objects leaving

› two people merging into one cluster, occlusions

Validation tip

Press v in the viewer: Renderer::getLidarStatus() becomes false and the tracker runs on **predictions only**. Tracks must keep moving straight and their uncertainty must grow.

Code structure



Main file, includes the functions and variables that calls the tracking functions (just change the path)

This class provides the object tracked info

TODO

Kalman filter implementation, contains the prediction and update functions

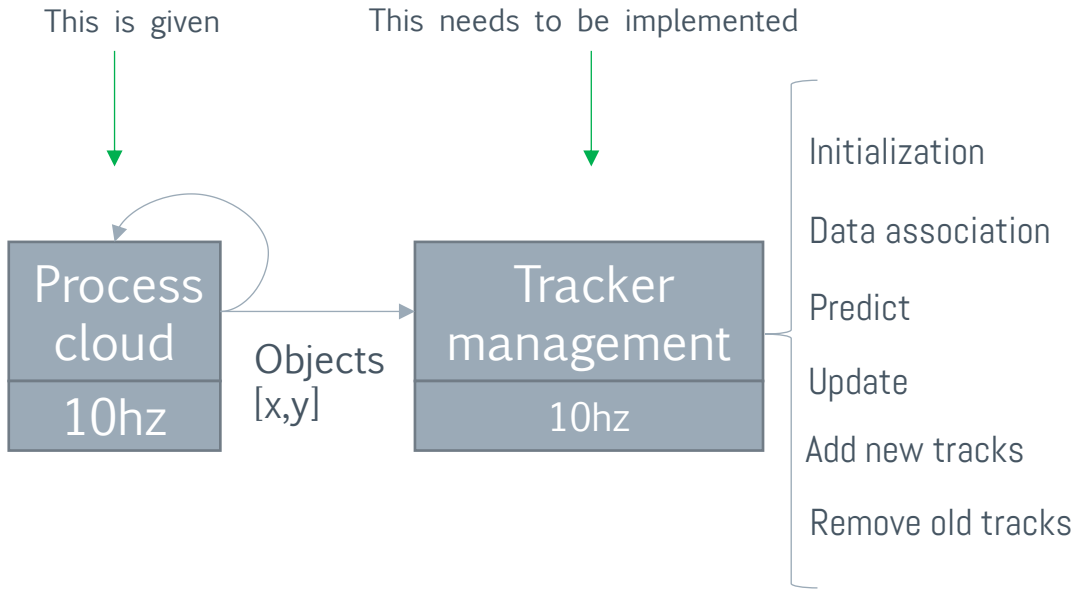
TODO

Tracking class includes the functions and variables that wraps the Kalman filter

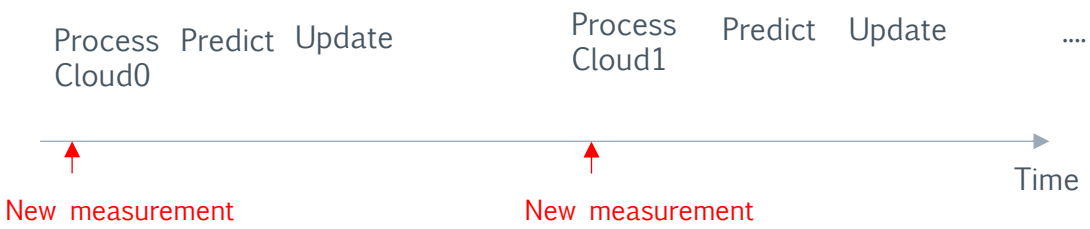
This class provides the functions to visualize the point cloud and objects tracked

This class contains the functions to process the point cloud. This function is called periodically (10hz as the LiDAR) and returns the position [x,y] of each object clustered

Main file



Execution example




```cpp
int main(int argc, char *argv[])
{
    int64_t freq = 100;            // Frequency of the thread dedicated to process the point cloud
    std::string log_path = "log";  // TODO: define the path to the log folder

    std::ifstream dataFile(log_path, std::ios::in | std::ios::binary);
    if (!dataFile)
    {
        std::cerr << "ERROR: The file '" << log_path << "' does not exist. Exiting.\n";
        return 1;
    }

    // Init renderer
    viewer::Renderer renderer;
    renderer.initCamera(viewer::CameraAngle::XY);
    renderer.clearViewer();

    // Instantiate the tracker
    Tracker tracker;

    // Spawn the thread that process the point cloud and performs the clustering
    CloudManager lidar_cloud(log_path, freq, renderer);
    std::thread t(&CloudManager::startCloudManager, &lidar_cloud);

    while (true)
    {
        // Clear the render
        renderer.clearViewer();

        while (!lidar_cloud.new_measurement)
            ; // wait for new data (we will execute the following code each 100ms)

        // fetch data
        lidar_cloud.mtxData.lock();
        auto cloud = lidar_cloud.getCloud();
        auto color = lidar_cloud.getColor();
        auto boxes = lidar_cloud.getBoxes();
        auto centroids_x = lidar_cloud.getCentroidsX();
        auto centroids_y = lidar_cloud.getCentroidsY();
        lidar_cloud.new_measurement = false;
        lidar_cloud.mtxData.unlock();

        // render pointcloud and boxes
        renderer.renderPointCloud(cloud, "pointCloud", color);
        for (size_t i = 0; i < boxes.size(); ++i)
            renderer.renderBox(boxes[i], i);

        // Call the tracker on the detected clusters
        tracker.track(centroids_x, centroids_y, renderer.getLidarStatus());

        // retrieve tracklets and render the trackers
        auto tracks = tracker.getTracks();
        for (size_t i = 0; i < tracks.size(); ++i)
        {
            renderer.addCircle(tracks[i].getX(), tracks[i].getY(), tracks[i].getId());
            renderer.addText(tracks[i].getX() + 0.01, tracks[i].getY() + 0.01, tracks[i].getId());
        }

        renderer.spinViewerOnce();
    }

    t.join();
    return 0;
}
```



KF Inizialization

- Initialize state x
- Initialize covariance P
- Initialize measurement covariance
- Initialize measurement matrix
- Initialize transition matrix
- Initialize noise and other variables that might be used

```
void KalmanFilter::init(double dt)
{
  dt_ = dt;

  // create a 4D state vector
  x_ = Eigen::VectorXd(4);

  // TODO: Initialize the state covariance matrix P
  P_ = Eigen::MatrixXd(4, 4);
  P_ << 9999., 0., 0., 0.,
        0., 9999., 0., 0.,
        0., 0., 9999., 0.,
        0., 0., 0., 9999.;

  // measurement covariance
  R_ = Eigen::MatrixXd(2, 2);
  R_ << 0.0225, 0.,
        0., 0.0225;

  // measurement matrix
  H_ = Eigen::MatrixXd(2, 4);
  H_ << 1., 0., 0., 0.,
        0., 1., 0., 0.;

  // the transition matrix F
  F_ = Eigen::MatrixXd(4, 4);
  F_ << 1., 0., dt_, 0.,
        0., 1., 0., dt_,
        0., 0., 1., 0.,
        0., 0., 0., 1.;

  // set the acceleration noise components
  double noise_ax_ = 2.;
  double noise_ay_ = 2.;

  double dt_2 = dt_ * dt_;
  double dt_3 = dt_2 * dt_;
  double dt_4 = dt_3 * dt_;

  // set the process covariance matrix Q
  Q_ = Eigen::MatrixXd(4, 4);
  Q_ << dt_4 / 4. * noise_ax_, 0., dt_3 / 2. * noise_ax_, 0.,
        0., dt_4 / 4. * noise_ay_, 0., dt_3 / 2. * noise_ay_,
        dt_3 / 2. * noise_ax_, 0., dt_2 * noise_ax_, 0.,
        0., dt_3 / 2. * noise_ay_, 0., dt_2 * noise_ay_;
}
```

KalmanFilter::predict() and update()

The equations of the lecture, written with the members of the class. Each line becomes one line of Eigen code

Step	Equation	Members involved	Meaning
predict	$x' = F\,x$	x_, F_	move the state with the motion model
	$P' = F\,P\,F^{T} + Q$	P_, F_, Q_	uncertainty grows
update	$y = z - H\,x$	z, H_, x_	innovation: the surprise
	$S = H\,P\,H^{T} + R$	H_, P_, R_	expected size of the surprise (2×2)
	$K = P\,H^{T}\,S^{-1}$	P_, H_, S	Kalman gain (4×2)
	$x' = x + K\,y$	x_	corrected state
	$P' = (I - K\,H)\,P$	I, P_	uncertainty shrinks; I = Identity(4, 4)

Eigen tips

- In Eigen, * is the matrix product (in numpy it would be @).
- Transpose: F_.transpose()
- Inverse: S.inverse() (S is only 2×2).
- Check sizes: x_ 4, P_ 4×4, H_ 2×4, S 2×2, K 4×2.
- Test with 'v': with predict only, circles move straight and P_ grows.

Kalman filter

- **Objective**: track the position of the objects
- The kalman filter can be used to predict their upcoming positions
- A constant velocity Kalman Filter is used to calculate the locations of the tracked objects coming from frame t, in the frame t+1
- Predict and update functions are used by the tracker manager to predict and update person's position

```
void KalmanFilter::predict()
{
  // TODO
  // Implement Kalman Filter Predict
  //  x_ = ...
  //  P_ = ...
}

void KalmanFilter::update(const Eigen::VectorXd &z)
{
  // TODO
  // Implement Kalman Filter Update

  // Eigen::VectorXd y = ...
  // Eigen::MatrixXd S = ...
  // Eigen::MatrixXd K = ...

  // new estimate
  // x_ = ...
  // Eigen::MatrixXd I = Eigen::MatrixXd::Identity(x_.size(), x_.size());
  // P_ = ...
}
```

Kalman filter design (Prediction)

x = estimate
P = uncertainty covariance
F = state transition matrix
u = motion vector
z = measurement
H = measurement function
R = measurement noise
I = identity matrix

prediction
$x' = Fx + u$
$P' = F \cdot P \cdot F^T$

- $x' = Fx + u$
  - Compute the state of x in the future
- $P' = FPF^T + Q$
  - Uncertainty about the state of the system
- Example: if we assume a system where we just assume the position in x – its corresponding velocity $\dot{x}$ and y –$\dot{y}$ resp.-
  - State vector x= $[x, y, \dot{x}, \dot{y}]$
  - Update matrix F = $\begin{pmatrix} 1 & 0 & \Delta_t & 0 \\ 0 & 1 & 0 & \Delta_t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$
  - We will assume the constant velocity model
    - Constant velocity means that the object in motion is moving in a straight line at a constant speed
    - New position→ p′ = p + vΔt
    - New velocity → v′=v
  - $\begin{pmatrix} 1 & 0 & \Delta_t & 0 \\ 0 & 1 & 0 & \Delta_t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ \dot{x} \\ \dot{y} \end{pmatrix} = \begin{pmatrix} px' = px + \dot{x}\Delta t \\ py' = py + \dot{y}\Delta t \\ vx' = \dot{x} \\ vy' = \dot{y} \end{pmatrix}$

Prediction

$$P' = \underbrace{FPF^T}_{\text{Deterministic}} + \underbrace{Q}_{\text{Stochastic}}$$

- P is the covariance matrix, holds the current uncertainty of the position and velocity
- We introduce to the model a random acceleration
  - Assume that this is a random value that models the stochastic part of the object
  - The **covariance Q** models the process noise: what the constant-velocity model cannot predict
    - We assume a random acceleration a, constant during Δt, with standard deviation σ_a

$$Q = \begin{pmatrix} \frac{\Delta t^4}{4}\sigma_{ax}^2 & 0 & \frac{\Delta t^3}{2}\sigma_{ax}^2 & 0 \\ 0 & \frac{\Delta t^4}{4}\sigma_{ay}^2 & 0 & \frac{\Delta t^3}{2}\sigma_{ay}^2 \\ \frac{\Delta t^3}{2}\sigma_{ax}^2 & 0 & \Delta t^2\sigma_{ax}^2 & 0 \\ 0 & \frac{\Delta t^3}{2}\sigma_{ay}^2 & 0 & \Delta t^2\sigma_{ay}^2 \end{pmatrix}$$

$\boldsymbol{\sigma}$ *represents the noise*

Initial P value

$$\boldsymbol{P} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & ? & 0 \\ 0 & 0 & 0 & ? \end{pmatrix}$$

Kalman filter design (Update)

UPDATE

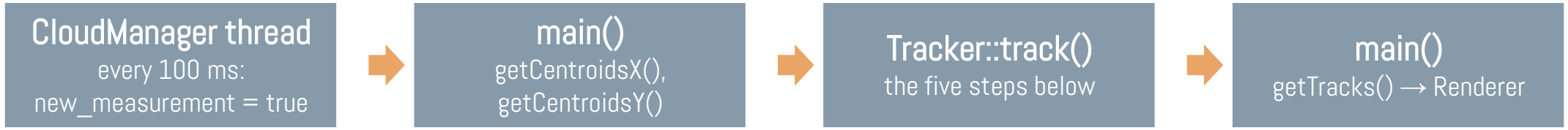
x = estimate
P = uncertainty covariance
F = state transition matrix
u = motion vector
z = measurement
H = measurement function
R = measurement noise
I = identity matrix

measurement update
$y = z - H \cdot x$
$S = H \cdot P \cdot H^T + R$
$K = P \cdot H^T \cdot S^{-1}$
$x' = x + (K \cdot y)$
$P' = (I - K \cdot H) \cdot P$

- $y = z - H * x$
  - Difference between the measured value and current state
- $S = HPH^T + R$
  - Used for computing the Kalman Gain
- $K = P * H^T * S^{-1}$
  - Kalman Gain is used to define a weight for the measurements
- $x' = x + (K * y)$
  - Update the system state
- $P' = (I - K * H) * P$
  - New covariance for the upcoming predict step

One frame, step by step

Every 100 ms CloudManager publishes new centroids and main() calls Tracker::track(), which runs five steps



1 · Predict
Tracklet::predict()
for every track

2 · Associate
dataAssociation()
detections ↔ tracks

3 · Update
Tracklet::update()
associated tracks only

4 · Remove
removeTracks()
lost tracks

5 · Add
addTracks()
new detections

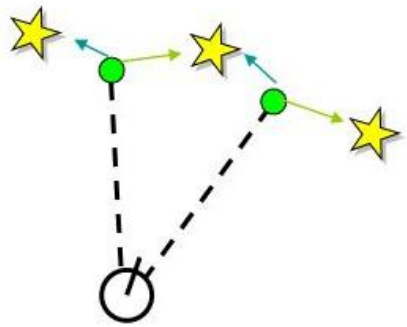
Timing assumption. Δt is fixed: kf_.init(0.1) in the Tracklet constructor. The log is assumed recorded at 10 Hz, and every point cloud is used exactly once. This is a **sequential** Kalman filter: one predict and one update per scan. Never compute Δt from the system clock

Tracklet: the life of a tracked object

Each Tracklet owns one KalmanFilter kf_, an id_ and a counter loss_count_ = frames since its last update

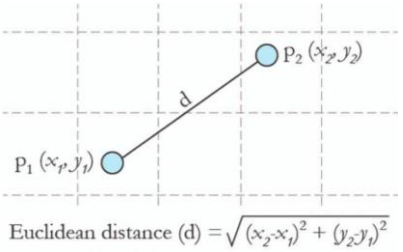
Created	Every frame	If associated	Removed
Tracklet(id, x, y) kf_.init(0.1) kf_.setState(x, y) loss_count_ = 0	predict() kf_.predict() loss_count_++	update(x, y, lidarStatus) kf_.update(z) loss_count_ = 0 (only if the LiDAR is on)	Tracker::removeTracks() too many frames lost or covariance too large

Tracker management (data association)

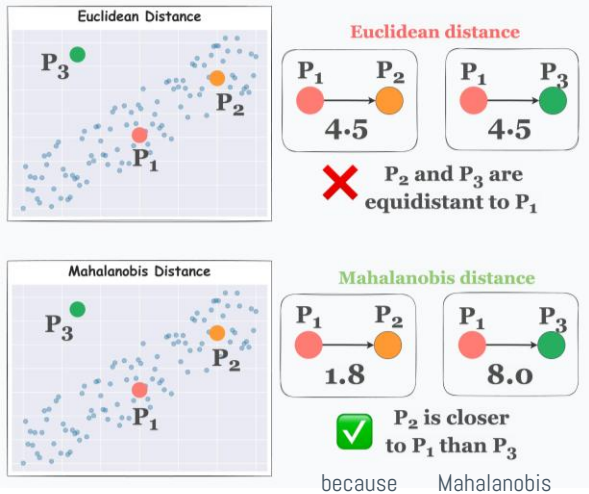


- In a tracking system based on the Kalman filter, each detected object is represented by a track, that is, a predicted state (position, velocity, etc.) and its uncertainty
  - At each time step, new measurements arrive from sensors, and the tracker must decide which measurement corresponds to which existing track
- **Data association: The process of assigning each measurement to an existing track, new or none**
  - **The tracker's task is to determine which measurement corresponds to which existing track, or whether a measurement should instead create a new track**
- **Problem**: multiple measurements can be assigned to different landmarks
- **Goal**: Associate new measurements with existing tracks, so that each Kalman filter update refines the right state estimate over time
  - Different approaches can be used

Tracker management (data association)



- In data association for tracking, the similarity between a predicted track and a new measurement can be evaluated using distance metrics. Two common choices in the literature:
  - Euclidean distance
    - Measures the straight-line distance between the predicted and observed points
    - Limitation: Ignores the correlation and uncertainty of the state variables.
    - Advantage: Simple and widely used.
  - Mahalanobis distance
    - Measures distance in **multivariate space**, taking into account the **covariance** (uncertainty and correlation) of the prediction
      - The Mahalanobis distance measures not only how far a measurement is from the predicted point, but also how likely that measurement is to belong to that prediction
    - Distance of the test point from the center of mass divided by the width of the ellipsoid in the direction of the test point
      - The lower the Mahalanobis Distance, the closer a point is to the set of benchmark points
- Mahalanobis distance accounts for the correlation and uncertainty (covariance) , while Euclidean distance treats all directions as equally reliable



because Mahalanobis distance accounts for the data distribution and correlations

Tracker management (new and old objects)

Different strategies can be used to manage the tracks:

- New objects
  - If a new detection enters the scene, it should be included in the array of tracks
    - So, if a detection is not associated, we assume is a new object to be tracked
- Old objects – i.e., objects that are out of the scene
  - Old objects shall be removed based on the covariance of the kalman filter
    - Covariance of x: P(0,0)
    - Covariance of y: P(1,1)
    - Remove the objects that has a covariance greater than an arbitrary number that you can discover by trial and error
  - Moreover, tracks that has not been associated an arbitrary number of frames shall be removed
    - See variable "lifeWithoutBeingAssociated" in the tracker class
    - Remove the objects that has a not been in the scene for a 'x' number of frames



Assignment objectives

Part	Points	What is evaluated
KF tracker compiles and works	0–8	tracking quality, timing (ms per frame of the tracker), code quality and comments, originality
README	2	scenarios and conclusions: the role of the covariances and thresholds (previous slide)
Extensions	0–5	one or more of the ideas below, or your own

Extension	Where to start in the code
Track that travelled the longest path (ID and length)	accumulate the distance in `Tracklet` at every update (use filtered positions)
Count the persons entering an area	after `tracker.getTracks()` in `main()`: check getX(), getY() against the area
Person that stayed longest in an area	store entry frame per `getId()`; 1 frame = 0.1 s
Heading (yaw) of each pedestrian	atan2(vy, vx): add a getter for x_[2], x_[3] in `KalmanFilter`. A yaw state needs a nonlinear model (EKF)
A tracking performance metric	see the optional slide: e.g. NIS from y and S in `update()`

Optional: measure your tracker

The log has **no ground truth**. How do you know that your tracker works, and that one tuning is better than another? Propose, implement and report a tracking performance metric.

What to deliver (in the README)

1. **Definition**
   what the metric measures and why it says something about tracking quality
2. **Implementation**
   how it is computed in your code
3. **Experiment**
   results for at least two configurations (e.g. different Q, R or gating threshold)
4. **Discussion**
   which configuration is better according to your metric, and its limits

Some ideas (you can propose your own)

Number of IDs
IDs created vs people actually visible: too many IDs = fragmented tracks
no ground truth needed

Track lifetime
average duration of the tracks: short tracks often mean lost or spurious objects
no ground truth needed

ID switches
count identity changes on a short segment, by visual inspection
no ground truth needed

Filter consistency (NIS)
mean of $y^T S^{-1} y$ over the updates: ≈ 2 for a 2D measurement if Q and R are well tuned
no ground truth needed

Association rate
% of detections associated to an existing track
no ground truth needed

Position error
label positions and IDs in a few frames by hand, then compute RMSE and ID errors
needs a few labelled frames

Particle filter preamble

Nacho Sañudo
HIPERT - University of Modena and Reggio Emilia
isanudo@unimore.it

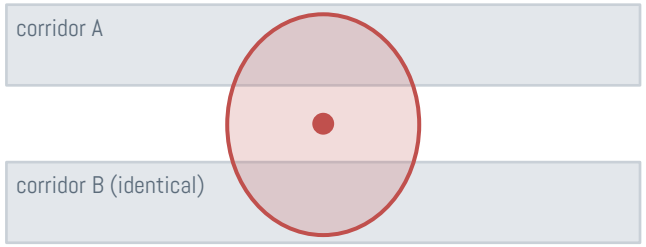
Academic year: 2026/2027

From tracking objects to localizing ourselves

We built a tracker for the objects around us. Can we use the same idea to track the **ego vehicle**, i.e. to know **where we are** in real time?

Object tracking (implemented using KF)	Ego localization (next lecture) … but
State: [x, y, vx, vy] of each pedestrian	**State:** our pose [x, y, θ] in a known map
Measurement: the cluster centroid, i.e. directly a position	**Measurement:** LiDAR distances to walls and reflectors, not our position
Linear model z = H x, Gaussian noise	1 **Nonlinear:** z = h(x, map): a small rotation changes all the distances a lot
One Gaussian per object is enough: the Kalman filter works	2 **Ambiguous:** symmetric corridors look the same: the belief has several peaks
	3 **Unknown start:** at power-on we could be anywhere in the map

One Gaussian (KF / EKF)



The mean falls between the corridors, where the robot cannot be.

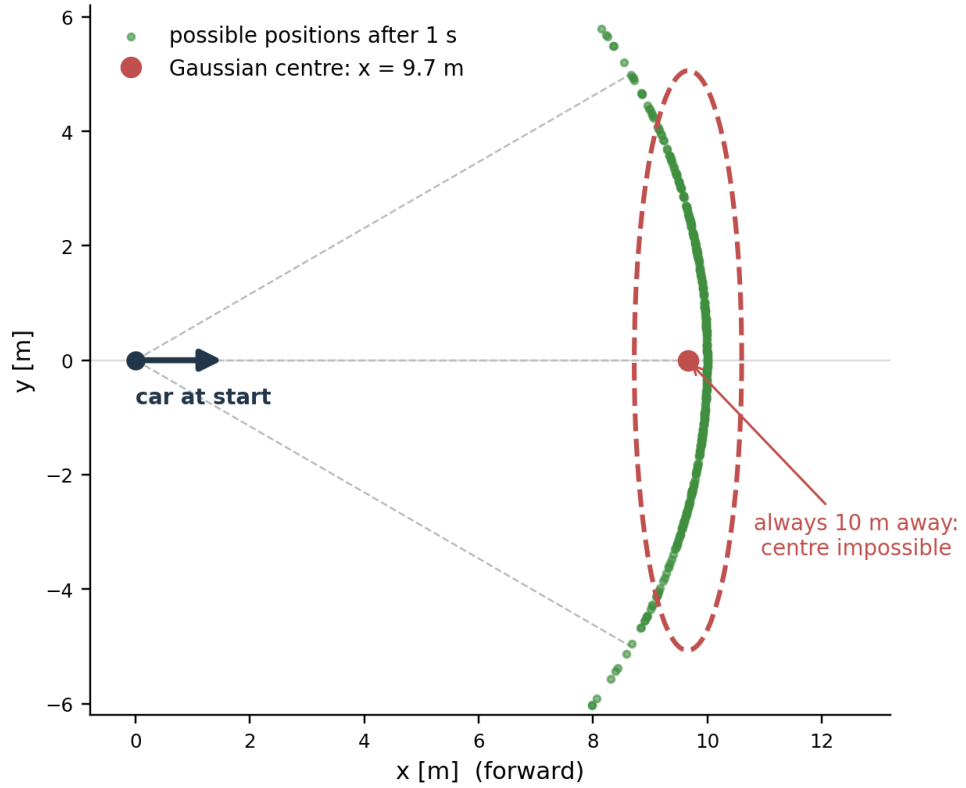
Many weighted hypotheses (Particle Filter)



Particles survive in both corridors until a measurement breaks the tie.

Particle Filter, which represents the uncertainty with many weighted hypotheses instead of a single Gaussian.

When the Kalman filter breaks



A car with an uncertain heading

Speed 10 m/s (known), heading 0° ± 15° (uncertain), Δt = 1 s

After 1 s the car is **always 10 m away**: the possible positions lie on an **arc**

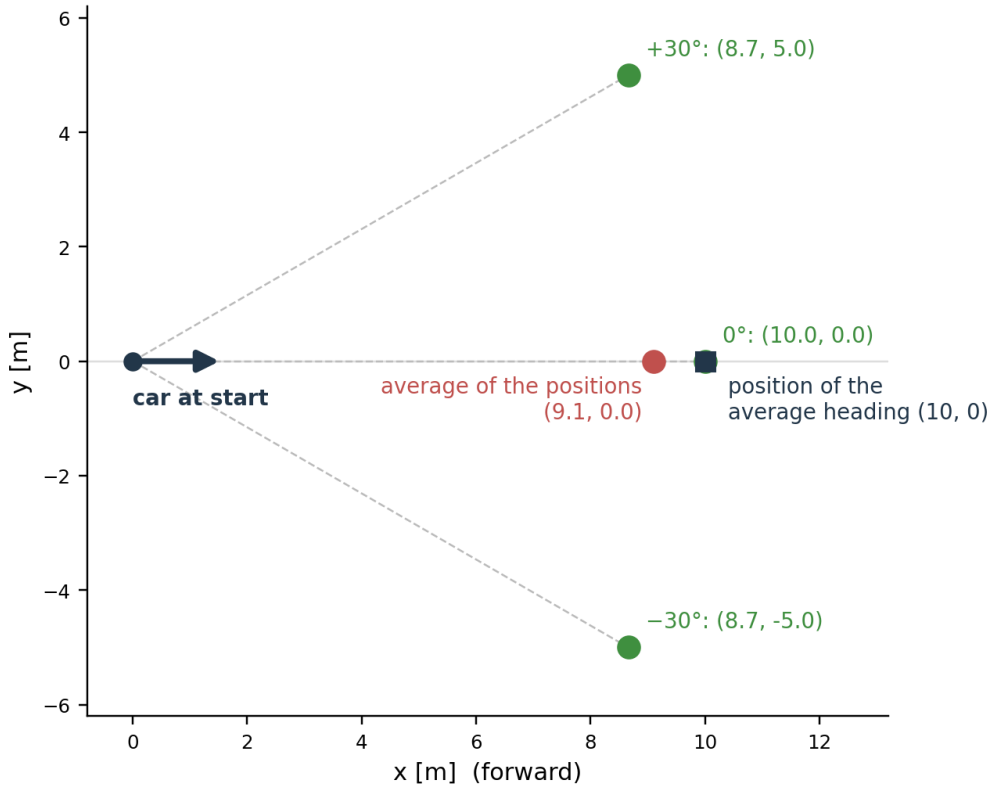
What the Kalman filter does

It can only describe the uncertainty as an **ellipse** (a Gaussian): a centre and a width

The best ellipse puts its centre at **x = 9.7 m**, inside the arc. To be there the car would have travelled only 9.7 m, but it **always travels 10 m**

The model bends the uncertainty, and an ellipse cannot bend

A bit of math: why it bends



The motion model

$$x' = x + v \cdot \Delta t \cdot \cos\theta$$
$$y' = y + v \cdot \Delta t \cdot \sin\theta$$

A **linear** model uses only + and × by constants, so it can be written as $x' = F \cdot x$. Here we apply cos and sin to a state variable: **no matrix F can do that**.

The rule to remember

$$f(\text{average}) \neq \text{average}(f)$$

Average heading 0° ⟶ (10, 0). Average of the positions ⟶ (9.1, 0). The KF uses the first one.

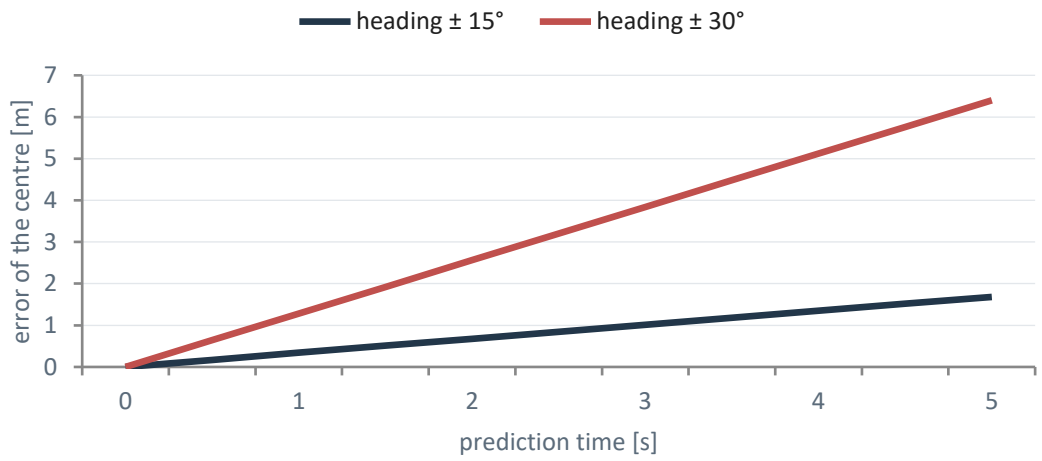
Typical nonlinear pieces: sin, cos, √, atan2, x·y, x².

Is a 30 cm error really a problem?

In one step, no. The problem is not the 30 cm. It is what happens **next**.

Curvature: how far the true curve moves away from its tangent, within the uncertainty.
±5° → 4 cm · ±15° → 34 cm · ±30° → 1.34 m. Same formula: almost straight with a small uncertainty, curved with a large one.

1 · The error grows with curvature and time



2 · The uncertainty P is wrong too

After linearizing, the filter believes the forward position has **no uncertainty** ($\sigma = 0$, true $\sigma = 0.47$ m).

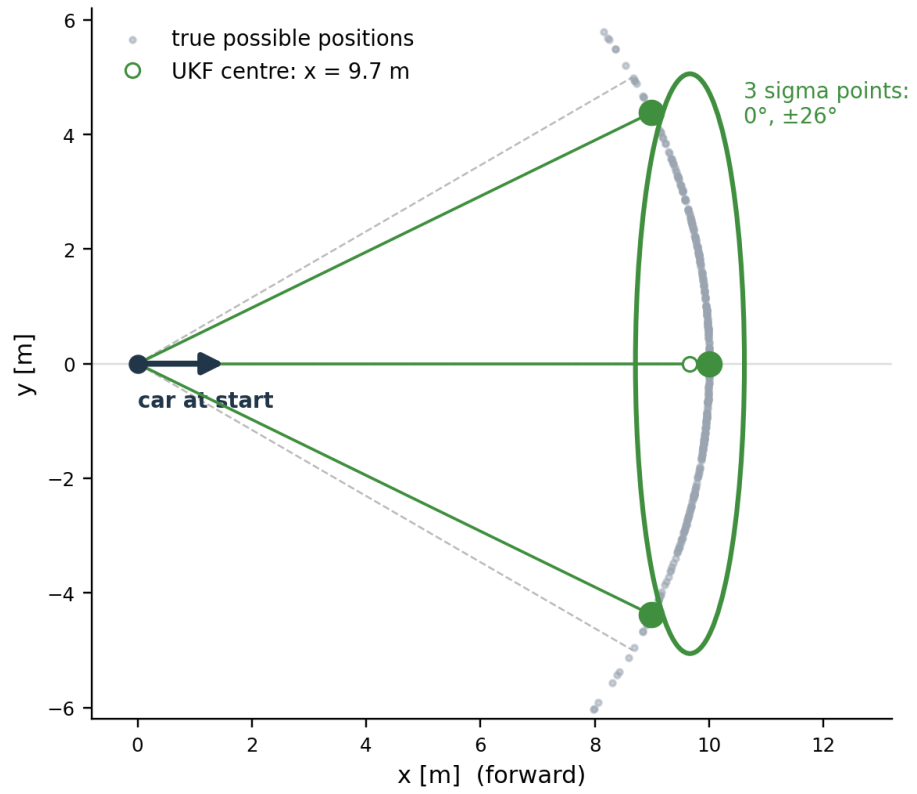
Overconfident: a measurement saying "the car is further back" is almost ignored.

Wrong gating: true detections at the ends of the arc can be rejected.

3 · The errors pile up

The filter is recursive: a wrong state with a too small P can make it **diverge**.

Solution 2 · UKF: move a few points



Idea

Do not touch the formula. Pick a few **representative points** (sigma points) and move them with the real formula.

Three steps

1 pick headings 0° and ±26°

2 compute x′ and y′ with the real cos and sin

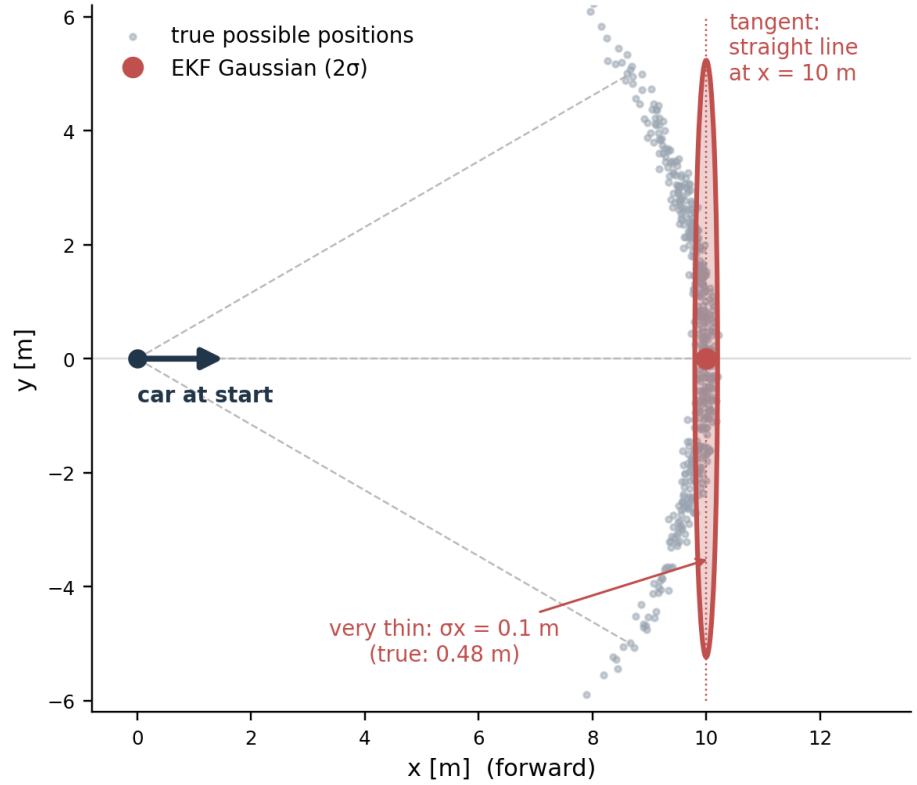
3 weighted mean and spread ⟶ the new ellipse

Result: centre **x = 9.7 m**, like the truth. With n state variables: 2n+1 points.

\+ no derivatives; follows the curvature

− the result is still an ellipse: no banana

Solution - EKF: pretend the curve is straight



Idea

Near the current estimate, replace the curve with its **tangent** (the derivative). Then use the normal KF

Linear again. In general the derivatives form the **Jacobian**, recomputed at every step

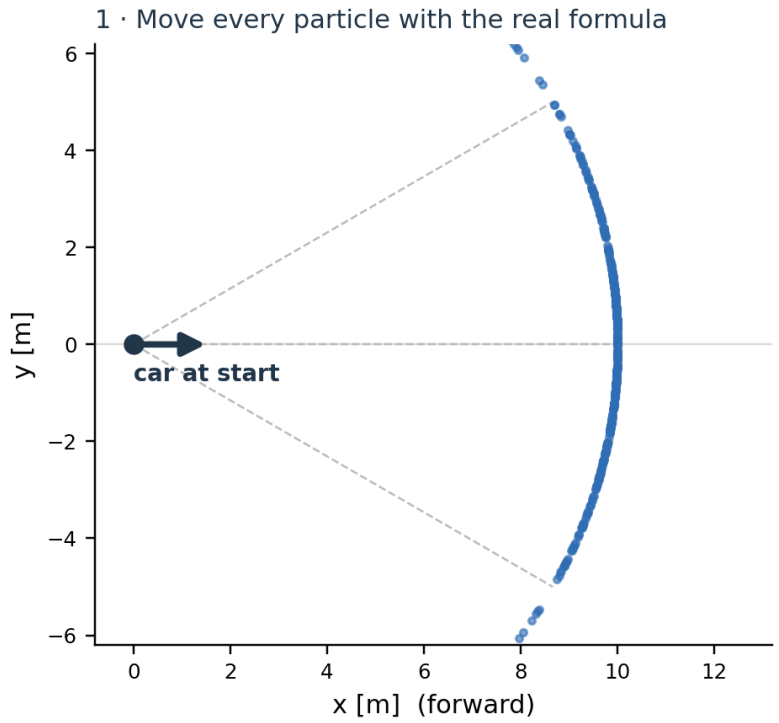
Result: centre **x = 10 m** and **no uncertainty forward** (true: 0.47 m). It ignores the curvature and is overconfident

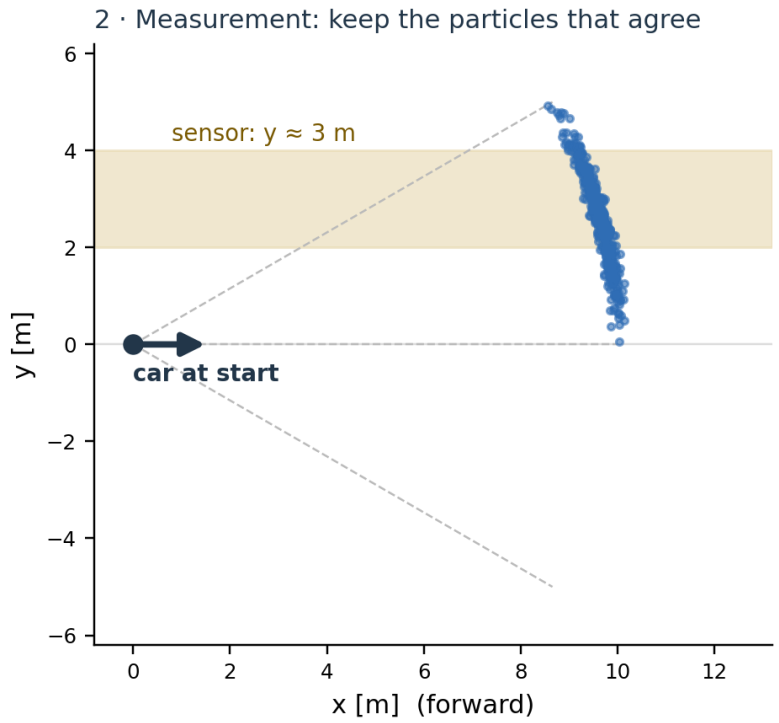
\+ fast: same cost as the KF

− wrong when the curvature is high; needs derivatives

Solution 3 · Particle Filter

Idea: no ellipse at all. Use hundreds of **particles**, each one a possible state, and keep them as they are.





At every step

1 move each particle

2 weight: does it agree with the sensor?

3 resample: keep the good ones

\+ keeps the real shape: banana, even two peaks

− hundreds or thousands of particles: slow